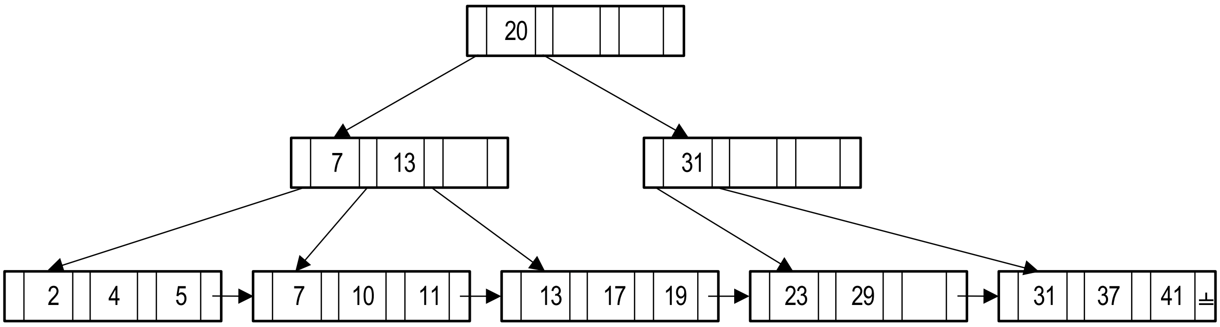
COMP 3311: Database Management Systems

Lecture 18 Exercises
Indexing: B$^+$-tree and Hash Index

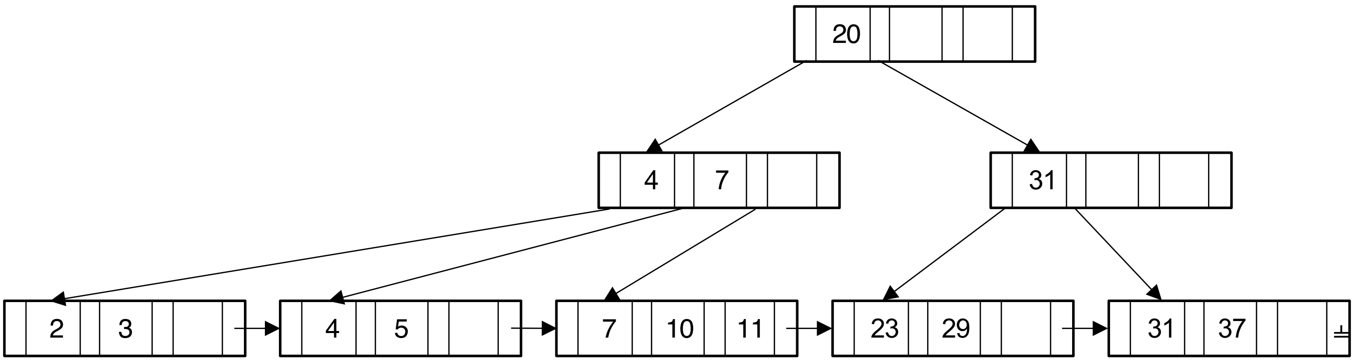
Exercise 1: Using initial the B$^+$-tree given below with fan out 4 and the B$^+$-tree insertion and deletion algorithms given in the lecture notes, show the B$^+$-tree that would result <u>after</u> performing the following operations *in the order given*.

i. insert 3   ii. insert 8



Exercise 2: Using initial the B$^+$-tree given below with and fan out 4 and the B$^+$-tree insertion and deletion algorithms given in the lecture notes, show the B$^+$-tree that would result <u>after</u> performing the following operations *in the order given*.

i. delete 5   ii. delete 3   iii. delete 7   iv. delete 11



Exercise 3: A company database has the following file and sizes of each field

Employee(employeeId: 6 bytes; employeeName: 10 bytes; departmentId: 4 bytes)

where departmentId is the id of the department where the employee works.

There are 100,000 employee records and 1,000 departments (each department has 100 employees).

A page is 1,000 bytes and a pointer is 4 bytes.

Assume that the file is sorted on departmentId and there is no index.

We want to build a hash index on employeeId on the above file where each entry has the form <employeeId, pointer>.

a) How many index entries are needed? (Briefly explain your answer.)

Index entries:

Explanation:

b) How many pages, $B_{employeeIdIndex}$, are required for these index entries (assuming full pages)?

$B_{employeeIdIndex}$:

c) Using the hash index, what is the page I/O cost of retrieving the record of an employee with a given employeeId, assuming that there are no overflow pages? (Briefly explain your answer.)

Page I/O cost:

Explanation:

Name: ________________________/_______________ Student#: ____________________ Date: ___________
Family/Given (PRINT) Given/First (PRINT)

This is an individual exercise. EACH STUDENT MUST SUBMIT THEIR OWN WORKSHEET.
A genuine effort must be made to complete all exercises to obtain full marks.

COMP 3311: Database Management Systems

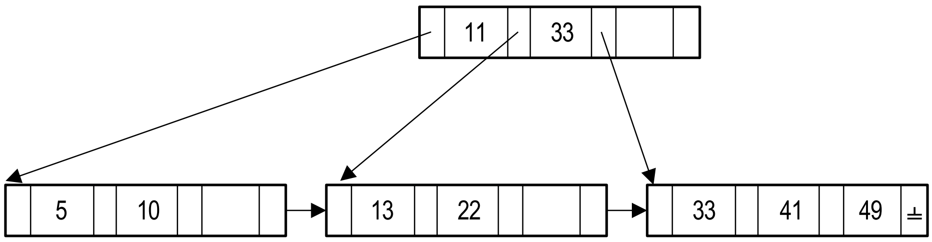
Lecture 18 Exercises
Indexing: B⁺-tree Index and Hash Index

Exercise 4: Using the initial B⁺-tree given below with fan out 4 and the B⁺-tree insertion and deletion algorithms given in the lecture notes, show the B⁺-tree that would result after performing the following operations *in the order given*. Add nodes to or cross out nodes in the empty B⁺-tree below as necessary.

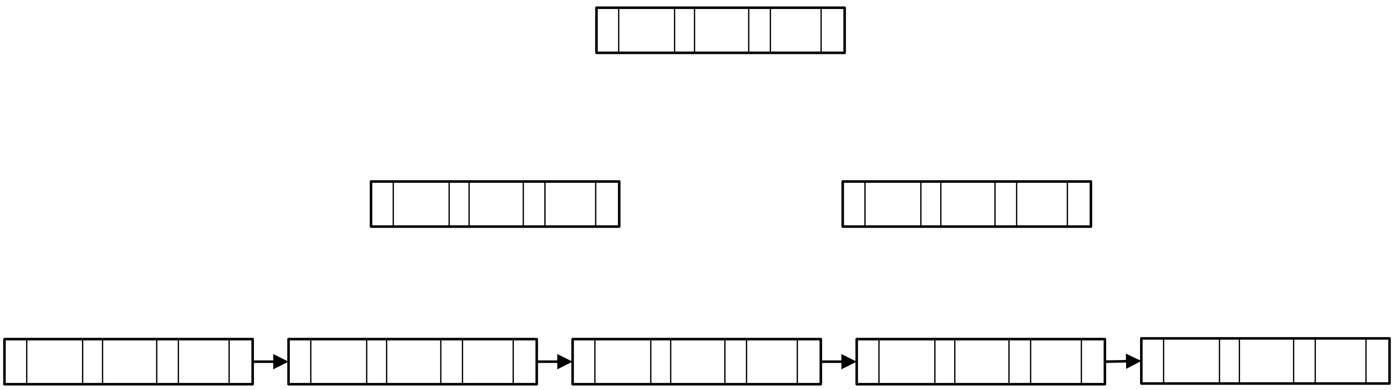
Perform the following operations *in the order given*

i. insert 45 ii. insert 35 iii. insert 40 iv. delete 49

Initial B⁺-tree



Final B⁺-tree



Name: ________________________/_______________ Student#: ____________________ Date: ___________
Family/Given (PRINT) Given/First (PRINT)

This is an individual exercise. EACH STUDENT MUST SUBMIT THEIR OWN WORKSHEET.
A genuine effort must be made to complete all exercises to obtain full marks.

COMP 3311: Database Management Systems

Lecture 18 Exercises
Indexing: B⁺-tree Index and Hash Index

Exercise 5: We want to construct a B+-tree for the primary key of an ordered, sequential file of 1,000,000 records, each 25 bytes, using bulk loading so that all B^+-tree nodes have minimum occupancy *where possible*. The primary key occupies 4 bytes. All pointers are 4 bytes. The page size is 128 bytes.

a) What is the blocking factor, $bf_{dataFile}$, of the data file?

$bf_{dataFile}$:

b) How many pages, $B_{dataFile}$, are needed for the data file?

$B_{dataFile}$:

c) What is the maximum number of values in each page of the B^+-tree index?

Maximum number of values:

d) What is the fan-out, *n*, of the B^+-tree?

Fan-out *n*:

e) What is the minimum occupancy (values) for the internal pages (nodes) of the B^+-tree?

f) What is the minimum occupancy (values) for the leaf pages (nodes) of the B^+-tree?

g) What is the height of the B^+-tree index and how many pages are there at each level of the B^+-tree?

Exercise 6: A global e-commerce website maintains a Customer file with attributes customerId, name, address, email and country. The file is organized as a hash file on customerId. There is also a secondary hash index on country. For each country there is only one index entry in the hash index. Assume that on average there are 12,000 customers for each country, there are 120 countries and that a page can hold 120 record pointers. How many page I/Os are required to retrieve the records of all the customers for a given country using the hash index on country? ***Explain your calculations; don't just give numbers.***